SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

QUESTIONS: **Total Marks: 50**

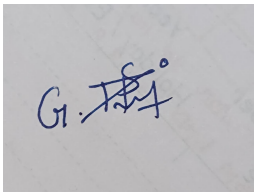
Annexure A

Debugging & Optimization Task

Code of Conduct:

*I**G.DHANUSHA SREE(192325012)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:G.DHANUSHA SREE



Annexure A
Debugging & Optimisation Task

1.Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.

B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

ANSWER:

1. Introduction

Symmetric encryption uses a secret key to convert readable plaintext into ciphertext and the corresponding key to recover the original data. DES (Data Encryption Standard) and 3DES (Triple DES) are historical symmetric-key block ciphers. DES processes 64-bit blocks and has an effective key length of 56 bits. 3DES applies DES operations multiple times to improve resistance to attacks, but it is slower and retains the 64-bit block size.

The task describes a Python implementation that produces inconsistent ciphertext for identical inputs across multiple runs. It asks for debugging of key scheduling and permutation steps, optimisation for datasets of at least 10 MB, and an assessment of whether modern ciphers improve security and performance. Since the actual source code is not supplied, this answer presents a systematic debugging approach rather than claiming that specific lines of code have been inspected.

2. Problem Statement

A Python implementation of DES and 3DES produces different ciphertext for the same plaintext and key across repeated runs. Possible causes include inconsistent key parsing, incorrect subkey generation, faulty initial/final permutations, incorrect bit indexing, mutable state leaking between calls, or an unintended changing IV or nonce. The implementation must be tested so that the same plaintext, key, mode, IV, and padding produce deterministic output where the selected mode is expected to do so. It must also process large inputs efficiently and be evaluated against modern cryptographic alternatives.

3. Objectives

Identify likely causes of non-deterministic ciphertext in DES/3DES code.

Validate key scheduling and permutation tables against the standard.

Use known-answer test vectors to confirm encryption and decryption correctness.

Improve throughput and memory use for datasets of 10 MB or more.

Compare DES/3DES with modern ciphers in terms of security, speed, and reliability.

Recommend a safe approach for new applications.

4. Understanding the DES Algorithm

4.1 DES structure

DES is a 16-round Feistel block cipher. It accepts a 64-bit input block and a 64-bit key representation, of which 8 bits are parity bits, leaving 56 effective key bits. The input passes through the Initial Permutation (IP), then splits into two 32-bit halves. Each round applies the round function to one half and combines the result with the other half. After 16 rounds, the halves are swapped and the Final Permutation (FP), which is the inverse of IP, is applied.

4.2 Key scheduling

DES key scheduling derives sixteen 48-bit round subkeys. The key first passes through Permuted Choice 1 (PC-1), selecting 56 bits and removing parity positions. The result is split into two 28-bit halves, C and D. Each round circularly left-shifts both halves according to the standard rotation schedule. Permuted Choice 2 (PC-2) then selects 48 bits from the combined halves to form that round's subkey.

The standard left-shift schedule for rounds 1–16 is: 1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1.

4.3 Permutation steps

The DES round function expands a 32-bit right half to 48 bits using the Expansion (E) table, XORs it with the 48-bit round subkey, passes the result through eight S-boxes to reduce 48 bits to 32 bits, and applies the P permutation. IP, E, S-boxes, P, PC-1, PC-2, and FP must match the published DES standard exactly.

A frequent implementation error is mixing the standard's one-based bit positions with Python's zero-based indexing. If a standard table contains position p, the Python index is usually p − 1. Other errors include reversing bit order, copying a permutation table incorrectly, applying IP or FP in the wrong direction, or changing the order of round subkeys.

5. Likely Causes of Inconsistent Ciphertext

5.1 Incorrect key parsing

The same key must always be converted into the same 64-bit representation. Common mistakes include interpreting a hexadecimal string as UTF-8 text in one function and as hexadecimal bytes in another, dropping leading zeroes, accepting different key

lengths without validation, or handling parity bits inconsistently. Define one key format and reject invalid input rather than silently guessing.

5.2 Faulty subkey generation

The PC-1 or PC-2 table, rotation schedule, or circular rotation may be wrong. A normal shift may drop bits instead of rotating a 28-bit half; a loop may use the wrong number of rotations; or the original key halves may be mutated and reused across calls. Encryption uses K1 through K16, while decryption uses the same subkeys in reverse order.

5.3 Incorrect permutation implementation

Permutation functions often contain off-by-one errors. A table may be edited incorrectly, a bit string may be reversed, or the output bits may be appended in the wrong order. The final permutation must be the inverse of the initial permutation. Published test vectors can identify these defects.

5.4 State leakage

Global or shared mutable variables may retain round halves, subkeys, or intermediate values between calls. Encryption state should be local to a call, and generated subkeys should not be modified unexpectedly. Run repeated tests in different orders to check for state leakage.

5.5 Mode, IV, padding, or randomness

Different modes have different expected behaviour. ECB with a fixed key and plaintext produces the same ciphertext. CBC produces the same ciphertext only if plaintext, key, IV, padding, and other parameters are the same. If a new random IV is generated for each run, different ciphertext is expected and is not necessarily a bug. The IV normally is not secret and must be stored or transmitted with the ciphertext when required.

First test the raw block cipher with a fixed known-answer test vector. Then test the mode separately. Do not remove random IV generation just to force identical outputs in a real system; that can weaken security.

6. Debugging and Corrective Actions

6.1 Validate input and key handling

Choose one key format, such as bytes or hexadecimal, and document it.

Validate key length and reject malformed input.

Use explicit conversion functions instead of implicit string conversion.

Handle DES parity consistently.

Keep key, plaintext, mode, IV, and padding fixed during reproducibility tests.

6.2 Verify key scheduling

Compare PC-1 and PC-2 tables against a trusted DES specification.

Confirm C and D are each 28 bits after PC-1.

Use circular left rotations that preserve all 28 bits.

Confirm the standard 16-round shift schedule.

Check that each round subkey is exactly 48 bits.

Use K1–K16 for encryption and K16–K1 for decryption.

6.3 Verify permutations and round function

Convert one-based positions to zero-based Python indexes consistently.

Check that IP and FP are inverses.

Check that E expands 32 bits to 48 bits and P maps 32 bits to 32 bits.

Verify S-box row and column extraction according to the DES definition.

Add assertions for expected bit lengths at important stages.

6.4 Use known-answer tests

A standard known-answer test supplies a known key and plaintext block and specifies the expected ciphertext. A commonly used DES vector is key 0x133457799BBCDFF1, plaintext 0x0123456789ABCDEF, and ciphertext 0x85E813540F0AB405. A correct implementation should reproduce the result when bit ordering and key representation are handled correctly.

Test both encryption and decryption. Encrypting and then decrypting should recover the original plaintext, but round-trip success alone is insufficient because two matching mistakes can cancel each other. Use both known-answer vectors and round-trip tests.

7. Debugging Checklist

Run the same fixed block through encryption ten times and compare outputs.

Run known-answer tests before and after optimisation.

Record mode, IV, padding, and library version in test logs; never log production secrets.

Test empty input, one block, multiple blocks, and data not aligned to the block size.

Test malformed keys and ciphertext and ensure the program fails safely.

Check supported 3DES keying options and reject weak configurations.

Restart the process and repeat a fixed test vector to detect initialisation or global-state problems.

8. Optimisation for Datasets of 10 MB or More

8.1 Use a vetted native cryptographic library

A major improvement is to replace handwritten Python bit-by-bit cryptographic operations with a well-maintained cryptographic library. Pure Python code that repeatedly creates strings, lists, and individual bit objects has high interpreter overhead. Native implementations written in optimized C or assembly can process blocks faster and are less likely to contain subtle algorithmic errors. Use the library API rather than rewriting cipher internals.

8.2 Process data in chunks

For large files, avoid loading the entire file into memory when the library supports incremental encryption. Read fixed-size chunks, update the cipher context, and write output incrementally. Chunk sizes such as 64 KiB or 1 MiB can be benchmarked. The selected mode and library must support streaming correctly, and padding and finalization must be handled as documented.

8.3 Avoid repeated setup work

Where the library permits a reusable context for the same key and operation, avoid rebuilding schedules or allocating temporary objects unnecessarily. Follow the library's context-reuse rules. For authenticated modes, preserve nonce uniqueness and authentication-tag validation.

8.4 Reduce conversions and copying

Repeated conversions between binary strings, hexadecimal strings, lists of integers, and byte arrays can dominate runtime. Use byte-oriented APIs, avoid converting the whole file to a string of zeroes and ones, and write results in chunks. Buffer interfaces can reduce unnecessary copying.

8.5 Benchmark correctly

Measure encryption and decryption separately using realistic files of at least 10 MB. Record elapsed time, throughput in MB/s, peak memory, and whether measurements include file I/O and padding. Warm up the code, repeat each test, and report the median or average along with the hardware, operating system, library, mode, and configuration. Do not invent speed figures: actual performance varies by platform and implementation.

9. Illustrative Efficient File-Processing Structure

The following is an illustrative structure, not a complete DES/3DES implementation. The cipher API must come from a vetted library, and the chosen mode must support incremental updates.

with open(input_path, 'rb') as source, open(output_path, 'wb') as target:

while True:

chunk = source.read(1024 * 1024) # 1 MiB chunk

if not chunk:

break

encrypted_chunk = cipher_context.update(chunk)

target.write(encrypted_chunk)

target.write(cipher_context.finalize())

Important: this pattern is valid only when the chosen library and mode support streaming updates and the program correctly handles padding, finalization, IVs or nonces, and authentication tags. With authenticated encryption, plaintext must not be trusted until authentication succeeds.

10. Comparison of DES, 3DES, and Modern Encryption

Criterion

DES

3DES

Modern AES-based option

- Security
- 56-bit effective key is inadequate
- Legacy; not recommended for new encryption
- AES-128/256 is widely approved when configured correctly
- Block size
- 64 bits
- 64 bits
- 128 bits
- Performance
- May be fast but insecure
- Usually slower due to multiple DES passes
- Often fast with optimised libraries and hardware support
- Integrity protection
- Basic encryption alone does not provide it
- Basic encryption alone does not provide it
- AES-GCM can provide confidentiality and integrity
- New project suitability
- Do not use
- Do not use for new systems
- Preferred when correctly configured and supported

11. Does Replacing DES/3DES Improve Security and Performance?

Yes. Replacing DES or 3DES with a modern, correctly implemented cipher generally improves security and can also improve performance. DES is obsolete because its effective key length is too short. 3DES is stronger than single DES but has a small block size, requires multiple DES operations, and is deprecated for new encryption. Modern AES implementations are highly optimized and may use processor instructions for accelerated encryption, so AES can be faster than 3DES on many current devices.

The performance claim must be qualified: speed depends on hardware, library, mode, and message size. On some small processors without AES acceleration, another approved modern option may perform better. Benchmark a vetted implementation rather than assuming a universal speed ratio.

For new applications, use authenticated encryption such as AES-GCM or another suitable, supported alternative. Authenticated encryption protects confidentiality and detects tampering. Correct design also requires unique nonces for modes such as GCM, secure key storage, key rotation, and safe error handling. Merely changing the algorithm will not correct faulty input handling unless the broken implementation is replaced or fixed.

12. Testing and Evaluation Plan

Testing and evaluation are important steps in debugging and optimizing the DES/3DES encryption implementation. The main purpose is to verify the correctness of encryption and decryption, identify logical errors, measure performance, and ensure that the program works reliably with large datasets. The following tests can be performed to evaluate the implementation.

1. Known-Answer Test

A known-answer test is used to verify the correctness of the DES algorithm by comparing the generated ciphertext with an expected ciphertext from a standard test vector.

Provide a predefined plaintext and encryption key.

Encrypt the plaintext using the DES implementation.

Compare the generated ciphertext with the expected ciphertext.

If both values match, the basic encryption operation is behaving correctly for that test vector.

If they differ, inspect key scheduling, permutation tables, bit ordering, and round operations.

For example, a standard DES test vector uses key 0x133457799BBCDFF1 and plaintext 0x0123456789ABCDEF. The expected ciphertext is 0x85E813540F0AB405.

This test helps identify errors in the core encryption algorithm.

2. Repeatability Test

The repeatability test checks whether the encryption program produces consistent results for identical inputs.

Select a fixed plaintext and encryption key.

Use the same encryption mode, initialization vector (IV), and padding settings.

Execute the encryption operation multiple times.

Compare the ciphertext generated during each execution.

Investigate any unexpected differences.

For a deterministic configuration such as ECB with fixed inputs, the ciphertext should remain identical. In CBC mode, the IV must also remain the same for this test. If the program generates a fresh IV for every run, different ciphertext is expected.

This test helps identify unintended randomness, incorrect initialization, and shared-state errors.

3. Round-Trip Test

The round-trip test verifies whether encryption and decryption work together correctly.

Select a sample plaintext and a valid encryption key.

Encrypt the plaintext to obtain ciphertext.

Decrypt the ciphertext using the corresponding key and correct parameters.

Compare the decrypted result with the original plaintext.

Confirm that the recovered data matches the original exactly.

If the decrypted data differs, check the round-key order, permutation operations, padding removal, and key handling.

This test verifies the complete encryption and decryption process. However, it should be combined with known-answer tests because matching errors in encryption and decryption may cancel each other out.

4. Invalid-Key Test

The invalid-key test checks how the program handles incorrect or unsupported encryption keys.

Provide a key with an invalid length or format.

Test malformed hexadecimal input, if hexadecimal keys are accepted.

Execute the encryption operation.

Verify that the program rejects invalid input and displays a clear error message.

Confirm that it does not silently modify or incorrectly interpret the key.

The program should validate keys before starting encryption. It should also handle unsupported 3DES keying configurations appropriately.

This test improves reliability and prevents unexpected encryption results caused by invalid key inputs.

5. Large-File Performance Test

The large-file performance test evaluates the efficiency of the encryption program when processing datasets of at least 10 MB.

Prepare test files of different sizes, such as 10 MB, 25 MB, and 50 MB.

Encrypt each file using the implementation

Record the execution time for every file.

Measure memory consumption during encryption.

Repeat the tests and calculate average or median execution time.

Compare the results before and after optimization.

The program can be optimized by using a well-maintained cryptographic library, processing data in chunks, and reducing unnecessary conversions between strings and byte arrays.

This test determines whether the implementation can process large datasets efficiently without excessive memory consumption.

6. Tampering Test

The tampering test evaluates whether unauthorized changes to encrypted data can be detected.

Encrypt a sample message using an authenticated encryption algorithm, such as AES-GCM.

Modify a small portion of the ciphertext or authentication tag.

Attempt to decrypt the modified data.

Verify that authentication fails and the program rejects the altered message.

Confirm that the application does not treat unauthenticated plaintext as valid.

Basic DES or 3DES encryption alone does not provide reliable integrity protection. Therefore, this test should be performed using an authenticated encryption implementation when evaluating the proposed modern replacement.

This test demonstrates the importance of protecting both the confidentiality and integrity of sensitive data.

7. Cross-Run Test

The cross-run test checks whether the encryption implementation behaves correctly after the program is restarted.

Run the program and encrypt a predefined test vector.

Record the ciphertext produced.

Close and restart the application.

Repeat the encryption using the same test inputs and parameters.

Compare the new result with the expected test-vector output.

Investigate differences caused by incorrect initialization, configuration changes, or global variables.

This test helps identify problems that may not appear when the program runs continuously.

8. Key-Scheduling Validation Test

Key scheduling generates the round keys used by DES during encryption and decryption. Errors in this process can produce incorrect ciphertext.

Verify that the PC-1 and PC-2 permutation tables match the DES specification.

Confirm that the original key is converted into the correct 56-bit effective key.

Check that the two key halves contain 28 bits each.

Validate the standard left-rotation schedule for all 16 rounds.

Confirm that every round key contains exactly 48 bits.

Verify that encryption uses the round keys in forward order and decryption uses them in reverse order.

This test helps locate errors in round-key generation and improves the correctness of the implementation.

9. Permutation Validation Test

DES uses several permutation operations to rearrange bits during encryption. Incorrect indexing or bit ordering can cause encryption failures.

Check the Initial Permutation (IP) table.

Verify that the Final Permutation (FP) is the inverse of IP.

Confirm that the Expansion (E) operation converts 32 bits into 48 bits.

Validate the P permutation used after the S-box operations.

Check that standard one-based bit positions are converted correctly to Python's zero-based indexes.

Compare intermediate results with trusted test vectors wherever possible.

This test ensures that the permutation steps follow the DES specification.

10. Decryption and Data-Integrity Test

This test checks whether the implementation can recover the original file without accidental loss or corruption.

Prepare a test file containing text, numbers, and binary data.

Encrypt the file using the selected mode.

Decrypt the resulting ciphertext.

Compare the original and recovered files byte by byte.

Test files of different sizes, including empty files and files that are not aligned to the cipher's block size.

Check padding and finalization behaviour.

The recovered file should match the original exactly when the correct key and parameters are used. Authenticated encryption should additionally verify the authentication tag before accepting the decrypted data.

11. Optimization Comparison Test

The optimization comparison test determines whether the revised implementation performs better than the original implementation.

Measure the original program's execution time and memory usage.

Replace inefficient bit-by-bit operations with a trusted cryptographic library where appropriate.

Apply chunk-based file processing to reduce memory usage.

Repeat the same tests using identical input files and hardware.

Compare encryption time, decryption time, throughput, and memory consumption.

Record the actual measurements without assuming a particular speed improvement.

The results can demonstrate whether the optimization reduces processing overhead and improves scalability for large datasets.

12. Final Security and Reliability Test

The final test evaluates the overall reliability and suitability of the encryption system.

Verify that all known-answer tests pass.

Confirm that invalid keys and malformed inputs are rejected safely.

Ensure that encryption and decryption recover the correct data.

Check that repeated deterministic test cases produce the expected ciphertext.

Verify that IVs and nonces are generated and managed according to the selected mode's requirements.

Confirm that authenticated encryption detects unauthorized ciphertext modification.

Evaluate performance using files of at least 10 MB.

Compare the implementation with a modern cipher such as AES-GCM.

Review the code for unnecessary data copying, unsafe error handling, and accidental exposure of secret keys.

Document test results, identified issues, corrections, and performance measurements.

This final evaluation helps determine whether the system is correct, efficient, and suitable for its intended purpose.

Conclusion

The testing and evaluation plan verifies the correctness of DES/3DES encryption, identifies errors in key scheduling and permutation operations, and measures performance for large datasets. Known-answer tests, repeatability tests, round-trip tests, and large-file tests are particularly important for diagnosing inconsistent ciphertext and evaluating optimization. Finally, comparison with a modern authenticated encryption algorithm such as AES-GCM helps determine whether migrating away from DES/3DES provides better security and practical performance.

13. Expected Outcomes

The cause of inconsistent output is isolated by testing the cipher, key schedule, permutation, and mode separately.

Key scheduling and permutation logic pass known-answer tests or reveal specific defects.

Fixed deterministic test inputs produce reproducible results.

Large-file processing uses bounded memory through chunked I/O where supported.

Performance is reported using measured results rather than unsupported speed claims.

The project recommends modern authenticated encryption for new systems.

14. Security and Implementation Considerations

Cryptographic code should not be designed only to make ciphertext repeatable. In modes requiring a random or unique IV/nonce, a new value may intentionally make ciphertext differ for repeated plaintext. The IV or nonce must be generated and managed according to the mode's rules and stored with the ciphertext when needed. Nonces must not be reused where the algorithm requires uniqueness. For authenticated encryption, authentication failure must cause decryption to fail safely.

DES and 3DES should be retained only when a documented legacy interoperability requirement makes them unavoidable. Isolate such functions, limit their exposure, and plan migration. Do not write a new DES/3DES implementation for production security. Use standard libraries, follow current guidance, and obtain independent review for sensitive deployments.

15. Conclusion

Inconsistent ciphertext from a DES/3DES program can arise from key-format errors, faulty PC-1/PC-2 or rotation logic, incorrect bit permutations, shared mutable state, or mode-level differences such as a changing IV. The correct debugging method is to validate each component against standard known-answer tests and test the mode separately. For files of 10 MB or more, a vetted native library, chunked file processing, fewer conversions, and repeatable benchmarks can improve speed and memory use. Replacing DES/3DES with a modern authenticated cipher such as AES-GCM generally improves security and often performance, although actual performance depends on the platform. The recommended solution is to debug systematically, validate with test vectors, benchmark honestly, and migrate new encryption to a modern supported design.

RUBRICS FOR CALCULATION

Criteria	Veigh ta ge	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identificati on	20%	Accurately identifies all errors and root causes with clear understandin g	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debuggin g approach; requires guidance	Unable to debug or resolve issues
ɔtimizati on	20%	Optimizes code by significantl y improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvemen t; inefficient approach retained
Analysis	20%	Demonstrate s strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrate d
Code Quality	15%	Produces clean, well-structured code with proper comments and documentatio n	Code is mostly clear with minor documentatio n gaps	Code lacks structure and proper documentati o n	Poorly written code with no documentati on